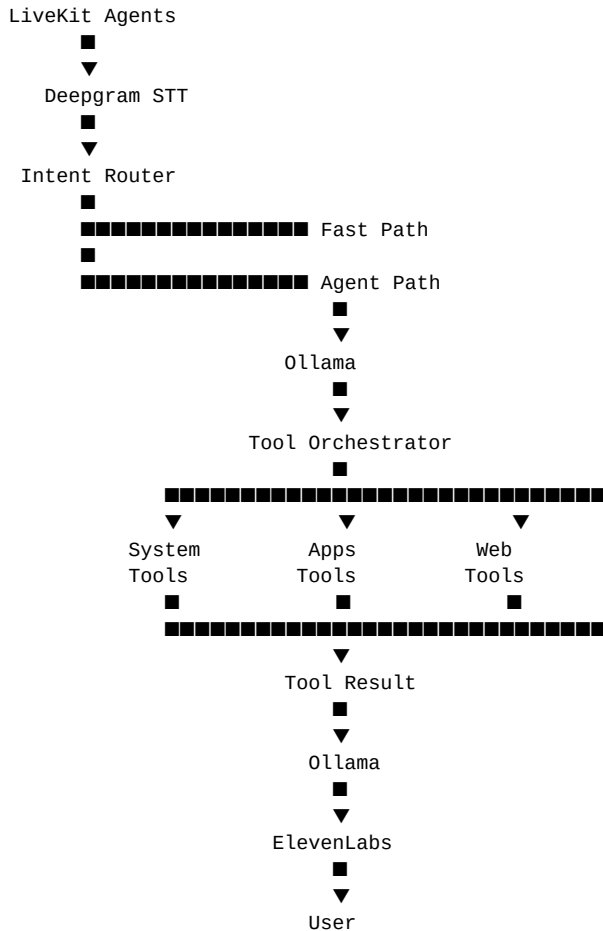


JARVIS Voice Assistant — Implementation Plan

Implementation roadmap for upgrading a LiveKit + Deepgram + ElevenLabs + Ollama voice bot into a real-time, context-aware AI assistant.



Phase 0 — Establish the Baseline

- Freeze the current working version.
- Measure STT, LLM, TTS, total response, and interruption latency.
- Measure Ollama inference speed and CPU/RAM/GPU usage.
- Add structured request tracing: USER_SPEECH → STT_FINAL → LLM_START → TOOL_CALL → TOOL_RESULT → LLM_FIRST_TOKEN → TTS_START → AUDIO_FIRST_BYTE → COMPLETE.
- Deliverable: identify exactly where latency is being spent.

Phase 1 — Refactor the Project

- Separate voice, agent, tools, memory, monitoring, security, and tests.
- Keep the existing voice bot working while responsibilities are separated.

Phase 2 — Build the Tool Registry

- Create ToolRegistry with name, description, input schema, permission level, handler, and timeout.

- Flow: LLM → Tool Call → Registry → Validation → Permission Check → Execution.
- First tools: `get_time`, `get_system_info`, `get_gpu_info`, `get_weather`, `media_control`.
- Deliverable: real tool execution instead of model hallucination.

Phase 3 — Build the Fast Path

- Create an `IntentRouter` with `DIRECT_TOOL`, `AGENT_TASK`, `CONVERSATION`, and `UNKNOWN` categories.
- Route simple commands directly to deterministic tools without unnecessary LLM reasoning.
- Cover time/date/weather/system status/GPU/media/open application first.

Phase 4 — Application Orchestrator

- Build `ApplicationRegistry`, `ApplicationResolver`, `ApplicationLauncher`, and `WindowManager`.
- Support aliases and fuzzy matching.
- Detect `NOT_RUNNING`, `RUNNING_BACKGROUND`, `RUNNING_FOREGROUND`, and `UNRESPONSIVE` states.
- Focus an existing app instead of launching a duplicate.
- Support launch arguments for projects/files.

Phase 5 — Media Controller

- Implement play, pause, next, previous, volume up/down, and mute.
- Add routing between Spotify, YouTube Music, and local media.
- Provide a configurable default media provider.

Phase 6 — System Monitoring

- Create a background `SystemMonitor`.
- Track CPU, GPU, RAM, VRAM, disk, network, temperature, and processes.
- Refresh every few seconds and maintain the latest system snapshot.

Phase 7 — Event Bus

- Create `EventBus` and `EventManager`.
- Generate events such as `GPU_HIGH_TEMP`, `CPU_HIGH_USAGE`, `DISK_FULL`, `NETWORK_DOWN`, `API_FAILURE`, `OLLAMA_DOWN`, and `LIVEKIT_DISCONNECTED`.

Phase 8 — Attention Manager

- Assign priority, severity, interruptibility, cooldown, and expiration to events.
- Queue low-priority events while JARVIS is speaking; interrupt only for important events.
- Track `USER_SPEAKING`, `JARVIS_SPEAKING`, `IDLE`, `EXECUTING_TASK`, and `AWAITING_APPROVAL` states.

Phase 9 — Emergency Failover

- Create `HealthManager` to track LiveKit, Deepgram, ElevenLabs, Ollama, internet, and external APIs independently.
- Use `HEALTHY`, `DEGRADED`, and `DOWN` states.

- Create ModelRouter for primary/local model fallback.
- Prevent one dependency failure from taking down the whole assistant.

Phase 10 — Memory

- Start with short-term conversation memory.
- Add working memory for current application, project, task, pending tool, and approval.
- Add long-term memory for useful preferences and frequently used locations.
- Use a Context Manager to provide only relevant memory to the LLM.

Phase 11 — Terminal and Filesystem

- Start with safe commands and read-only filesystem operations.
- Add file search, listing, reading, copying, moving, directory creation, PDF extraction, and summarization.
- Avoid unrestricted shell access initially.

Phase 12 — Approval System

- Create PermissionManager and ApprovalManager.
- Classify operations as READ_ONLY, LOW_RISK, MODIFY, DESTRUCTIVE, or EXTERNAL_ACTION.
- Require explicit confirmation for destructive and external actions.
- Bind approval to the specific pending operation.

Phase 13 — Agent Planner

- Create Planner, Executor, and TaskState.
- Convert multi-step goals into a task graph.
- Example: locate project → check Python → check GPU → check dependencies → open project → report.

Phase 14 — Recovery / Self-Healing

- Implement bounded retries, timeouts, fallbacks, and failure reasons.
- Allow deterministic recovery such as finding an alternative executable.
- Never permit unlimited agent retries.

Phase 15 — Weather / Context

- Add weather, forecast, AQI, sunset, and local time tools.
- Use context enrichment only when relevant instead of calling context tools for every request.

Phase 16 — Proactive Assistant

- Combine monitoring, EventBus, AttentionManager, memory, and task management.
- Allow critical system alerts, service failure notifications, and useful proactive reminders.

Phase 17 — Email / Calendar Workflows

- Use prepare → approve → execute workflows.
- Gather relevant information, prepare drafts or calendar changes, then request approval before external actions.

Phase 18 — Optimize the Voice Experience

- Optimize for time-to-first-audio rather than total generation time.
- Use streaming STT, early intent detection, streaming LLM output, sentence-level TTS, and interruption cancellation.
- Target fast perceived response rather than waiting for complete generation.

Phase 19 — Observability

- Track STT, router, LLM, tool, TTS, and total latency.
- Track tool success/failure, model used, fallback rate, false tool calls, and interruptions.
- Add average, p95, and p99 latency metrics.

Recommended Implementation Order

```

Phase 0: Baseline
↓
Phase 1: Architecture
↓
Phase 2: Tool Registry
↓
Phase 3: Fast Path
↓
Phase 4-5: Apps + Media
↓
Phase 6-8: Monitoring + Events + Attention
↓
Phase 9: Failover
↓
Phase 10: Memory
↓
Phase 11-12: OS + Security
↓
Phase 13-14: Agent + Recovery
↓
Phase 15-17: Context + Proactive Tasks
↓
Phase 18-19: Optimization + Observability

```

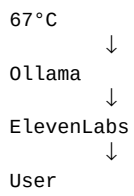
First Major Milestone

Do not start with the planner or a large memory system. First make the core loop rock-solid: JARVIS hears a command, understands it, selects a tool, safely executes it, receives the result, and speaks the result back while supporting streaming and interruption.

```

User:
"What's my GPU temperature?"
↓
Deepgram
↓
Intent Router
↓
gpu_status()
↓
Tool Registry
↓
Permission Check
↓
GPU Tool
↓

```



Once this loop is reliable, every new capability becomes a tool rather than requiring another redesign of the voice bot.